Tugas Pertemuan 1	
IF21W0508 – Praktikum Web	
NPM	243040093
Nama	Akbar putra pamungkas
Kelas	E
Link Github Repository	https://github.com/akbarputrapamungkas/PW2026_243040093/tree/main/pertemuan2

Bagian 1. Screenshot Semua Kode
1. App.jsx

```jsx
// app.jsx
import { useState, useEffect } from "react";
import "./App.css";
import Kartu from "./Kartu";
import Penghitung from "./penghitung";
import FormTugas from "./FormTugas";
import ItemTugas from "./ItemTugas";
import CatatanKeuangan from "./CatatanKeuangan";

function App() {
  // State untuk berpindah tab antara Tugas 1 & 2 (Todo) dan Tugas 3 (Catatan Keuangan)
  const [tabAktif, setTabAktif] = useState("todo");

  // 1. State utama: ambil dari localStorage atau gunakan data contoh
  const [daftarTugas, setDaftarTugas] = useState(() => {
    const dataTersimpan = localStorage.getItem("daftar-tugas");
    if (dataTersimpan) {
      return JSON.parse(dataTersimpan);
    }
    return [
      { id: 1, judul: "Install Node.js dan Vite", selesai: true },
      { id: 2, judul: "Pelajari useState", selesai: false },
    ];
  });

  // State baru untuk fitur filter ('semua', 'belum', 'selesai')
  const [filter, setFilter] = useState("semua");

  // Efek A: dijalankan SEKALI saat App pertama kali tampil
  useEffect(() => {
    console.log("Aplikasi Praktikum Modul 2 siap digunakan!");
  }, []);

  // Efek B: perbarui judul tab peramban setiap daftar tugas berubah
  useEffect(() => {
    const sisa = daftarTugas.filter((tugas) => !tugas.selesai).length;
    document.title = `${sisa} Tugas Belum Selesai | Praktikum PW`;
  }, [daftarTugas]);

  // Efek C: simpan daftarTugas ke localStorage setiap kali berubah
  useEffect(() => {
    localStorage.setItem("daftar-tugas", JSON.stringify(daftarTugas));
  }, [daftarTugas]);

  // 2. Menambah tugas
  const tambahTugas = (judul) => {
    const tugasBaru = { id: Date.now(), judul: judul, selesai: false };
    setDaftarTugas([...daftarTugas, tugasBaru]);
  };

  // 3. Mengubah status selesai
  const ubahStatus = (id) => {
    const daftarBaru = daftarTugas.map((tugas) => {
      if (tugas.id === id) {
        return { ...tugas, selesai: !tugas.selesai };
      }
      return tugas;
    });
    setDaftarTugas(daftarBaru);
  };

  // 4. Menghapus tugas berdasarkan ID
  const hapusTugas = (id) => {
    setDaftarTugas(daftarTugas.filter((tugas) => tugas.id !== id));
  };

  // Latihan 2: Fungsi untuk menghapus semua tugas yang sudah selesai
  const hapusTugasSelesai = () => {
    setDaftarTugas(daftarTugas.filter((tugas) => !tugas.selesai));
  };

  // Latihan 1: Logika memfilter tugas berdasarkan state 'filter'
  const daftarTampil = daftarTugas.filter((tugas) => {
    if (filter === "belum") return !tugas.selesai;
    if (filter === "selesai") return tugas.selesai;
    return true; // 'semua'
  });

  const jumlahSelesai = daftarTugas.filter((tugas) => tugas.selesai).length;

  return (
    <div className="wadah">
      <h1 className="judul">Praktikum Modul 2: React Lanjutan</h1>
      <p className="subjudul">
        Laboratorium Informatika - Manipulasi State, Conditional Rendering & LocalStorage
      </p>

      {/* Navigasi Tab */}
      <div
        style={{
          display: "flex",
          gap: "8px",
          justifyContent: "center",
          marginBottom: "20px",
        }}
      >
        <button
          onClick={() => setTabAktif("todo")}
          style={{
            padding: "8px 16px",
            borderRadius: "8px",
            cursor: "pointer",
            fontWeight: tabAktif === "todo" ? "bold" : "normal",
            background: tabAktif === "todo" ? "#4f46e5" : "#e5e7eb",
            color: tabAktif === "todo" ? "white" : "#374151",
            border: "none",
          }}
        >
          Tugas 1 & 2 (Todo)
        </button>
        <button
          onClick={() => setTabAktif("keuangan")}
          style={{
            padding: "8px 16px",
            borderRadius: "8px",
            cursor: "pointer",
            fontWeight: tabAktif === "keuangan" ? "bold" : "normal",
            background: tabAktif === "keuangan" ? "#4f46e5" : "#e5e7eb",
            color: tabAktif === "keuangan" ? "white" : "#374151",
            border: "none",
          }}
        >
          Tugas 3 (Catatan Keuangan)
        </button>
      </div>

      {tabAktif === "todo" ? (
        <>
          <Kartu>
            <FormTugas onTambah={tambahTugas} />
          </Kartu>

          <Kartu>
            <Penghitung
              total={daftarTugas.length}
              selesai={jumlahSelesai}
              sisa={daftarTugas.length - jumlahSelesai}
            />
          </Kartu>

          {/* Tombol Filter */}
          <div
            style={{
              display: "flex",
              gap: "8px",
              justifyContent: "center",
              marginBottom: "16px",
            }}
          >
            <button
              onClick={() => setFilter("semua")}
              style={{
                fontWeight: filter === "semua" ? "bold" : "normal",
              }}
            >
              Semua
            </button>
            <button
              onClick={() => setFilter("belum")}
              style={{
                fontWeight: filter === "belum" ? "bold" : "normal",
              }}
            >
              Belum Selesai
            </button>
            <button
              onClick={() => setFilter("selesai")}
              style={{
                fontWeight: filter === "selesai" ? "bold" : "normal",
              }}
            >
              Selesai
            </button>
          </div>

          <Kartu>
            <ItemTugas
              daftarTugas={daftarTampil}
              onUbahStatus={ubahStatus}
              onHapus={hapusTugas}
            />
          </Kartu>

          {/* Tombol Hapus Semua Selesai */}
          {jumlahSelesai > 0 && (
            <div
              style={{
                textAlign: "center",
                marginTop: "16px",
              }}
            >
              <button
                onClick={hapusTugasSelesai}
                style={{
                  background: "#ef4444",
                  color: "white",
                  padding: "8px 16px",
                  borderRadius: "8px",
                  cursor: "pointer",
                  border: "none",
                }}
              >
                Hapus Semua yang Selesai
              </button>
            </div>
          )}
        </>
      ) : (
        <Kartu>
          <CatatanKeuangan />
        </Kartu>
      )}
    </div>
  );
}

export default App;
```

2. CatatanKeuangan.jsx

```jsx
// src/CatatanKeuangan.jsx
import { useState, useEffect } from "react";
import Kartu from "./Kartu";
import FormTransaksi from "./FormTransaksi";
import DaftarTransaksi from "./DaftarTransaksi";
import RingkasanSaldo from "./RingkasanSaldo";

function CatatanKeuangan() {
  // Kriteria (f): Data tersimpan di localStorage
  // 1. Inisialisasi state dari localStorage (atau data default jika belum ada)
  const [transaksi, setTransaksi] = useState(() => {
    const dataTersimpan = localStorage.getItem("catatan-keuangan-mini");
    if (dataTersimpan) {
      try {
        return JSON.parse(dataTersimpan);
      } catch (err) {
        console.error("Gagal mengurai data localStorage:", err);
      }
    }
    // Data bawaan awal agar tampilan tidak kosong saat pertama dibuka
    return [
      {
        id: 1,
        keterangan: "Gaji Bulanan / Uang Saku",
        nominal: 1500000,
        jenis: "pemasukan",
        tanggal: "01 Nov 2026",
      },
      {
        id: 2,
        keterangan: "Beli Buku Modul Praktikum",
        nominal: 85000,
        jenis: "pengeluaran",
        tanggal: "03 Nov 2026",
      },
    ];
  });

  // 2. useEffect: Simpan data transaksi ke localStorage setiap kali state `transaksi` berubah
  useEffect(() => {
    localStorage.setItem("catatan-keuangan-mini", JSON.stringify(transaksi));
  }, [transaksi]);

  // Handler: Tambah transaksi baru (immutable state update)
  const tambahTransaksi = (transaksiBaru) => {
    setTransaksi([transaksiBaru, ...transaksi]);
  };

  // Handler: Hapus transaksi berdasarkan ID
  const hapusTransaksi = (id) => {
    setTransaksi(transaksi.filter((item) => item.id !== id));
  };

  return (
    <div className="modul-keuangan">
      {/* Ringkasan Saldo Otomatis */}
      <Kartu judul="Ringkasan Saldo">
        <RingkasanSaldo daftarTransaksi={transaksi} />
      </Kartu>

      {/* Formulir Input Transaksi */}
      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambahTransaksi={tambahTransaksi} />
      </Kartu>

      {/* Daftar Transaksi */}
      <Kartu judul="Daftar Transaksi">
        <DaftarTransaksi
          daftarTransaksi={transaksi}
          onHapusTransaksi={hapusTransaksi}
        />
      </Kartu>
    </div>
  );
}

export default CatatanKeuangan;
```

3. FormTransaksi.jsx

```jsx
// src/FormTransaksi.jsx
import { useState } from "react";

function FormTransaksi({ onTambahTransaksi }) {
  // Formulir terkendali (controlled form): state untuk setiap input
  const [keterangan, setKeterangan] = useState("");
  const [nominal, setNominal] = useState("");
  const [jenis, setJenis] = useState("pemasukan"); // 'pemasukan' atau 'pengeluaran'
  const [pesanGalat, setPesanGalat] = useState("");

  const tanganiSubmit = (e) => {
    e.preventDefault();

    // Validasi sederhana: pastikan keterangan dan nominal terisi
    if (keterangan.trim() === "" || nominal === "") {
      setPesanGalat("Keterangan dan nominal wajib diisi!");
      return;
    }

    // Petunjuk Modul 2: Nilai input selalu berupa teks, ubah menjadi angka
    const nilaiNominal = Number(nominal);
    if (isNaN(nilaiNominal) || nilaiNominal <= 0) {
      setPesanGalat("Nominal harus berupa angka lebih besar dari 0!");
      return;
    }

    // Buat objek transaksi baru
    const transaksiBaru = {
      id: Date.now(),
      keterangan: keterangan.trim(),
      nominal: nilaiNominal,
      jenis: jenis,
      tanggal: new Date().toLocaleDateString("id-ID", {
        day: "2-digit",
        month: "short",
        year: "numeric",
      }),
    };

    // Kirim data ke komponen induk
    onTambahTransaksi(transaksiBaru);

    // Reset nilai formulir
    setKeterangan("");
    setNominal("");
    setJenis("pemasukan");
    setPesanGalat("");
  };

  return (
    <form onSubmit={tanganiSubmit} className="form-keuangan">
      {pesanGalat && <p className="galat">{pesanGalat}</p>}

      <div className="form-grup">
        <label>Keterangan</label>
        <input
          type="text"
          className="input"
          placeholder="Contoh: Gaji Bulanan, Makan Siang"
          value={keterangan}
          onChange={(e) => setKeterangan(e.target.value)}
        />
      </div>

      <div className="form-baris">
        <div className="form-grup" style={{ flex: 2 }}>
          <label>Nominal (Rp)</label>
          <input
            type="number"
            className="input"
            placeholder="Contoh: 50000"
            value={nominal}
            onChange={(e) => setNominal(e.target.value)}
          />
        </div>

        <div className="form-grup" style={{ flex: 1 }}>
          <label>Jenis Transaksi</label>
          <select
            className="input"
            value={jenis}
            onChange={(e) => setJenis(e.target.value)}
          >
            <option value="pemasukan">Pemasukan (+)</option>
            <option value="pengeluaran">Pengeluaran (-)</option>
          </select>
        </div>
      </div>

      <button type="submit" className="tombol tombol-simpan">
        + Tambah Transaksi
      </button>
    </form>
  );
}

export default FormTransaksi;
```

4. RingkasanSaldo.jsx

```jsx
// src/RingkasanSaldo.jsx

function RingkasanSaldo({ daftarTransaksi }) {
  // Kriteria (c): Total saldo yang dihitung otomatis dari state (Derived State)
  const totalPemasukan = daftarTransaksi
    .filter((t) => t.jenis === "pemasukan")
    .reduce((total, t) => total + t.nominal, 0);

  const totalPengeluaran = daftarTransaksi
    .filter((t) => t.jenis === "pengeluaran")
    .reduce((total, t) => total + t.nominal, 0);

  const totalSaldo = totalPemasukan - totalPengeluaran;

  const formatRupiah = (angka) => {
    return new Intl.NumberFormat("id-ID", {
      style: "currency",
      currency: "IDR",
      maximumFractionDigits: 0,
    }).format(angka);
  };

  return (
    <div className="kartu-ringkasan-saldo">
      <div className="saldo-utama">
        <span>Total Saldo Saat Ini</span>
        <h3 className={totalSaldo >= 0 ? "teks-hijau" : "teks-merah"}>
          {formatRupiah(totalSaldo)}
        </h3>
      </div>

      <div className="detail-arus-kas">
        <div className="box-arus pemasukan">
          <span>Total Pemasukan</span>
          <strong className="teks-hijau">+{formatRupiah(totalPemasukan)}</strong>
        </div>
        <div className="box-arus pengeluaran">
          <span>Total Pengeluaran</span>
          <strong className="teks-merah">-{formatRupiah(totalPengeluaran)}</strong>
        </div>
      </div>
    </div>
  );
}

export default RingkasanSaldo;
```

5. ItemTransaksi.jsx



```jsx
// src/ItemTransaksi.jsx

function ItemTransaksi({ transaksi, onHapus }) {
  // Format angka ke mata uang Rupiah
  const formatRupiah = (angka) => {
    return new Intl.NumberFormat("id-ID", {
      style: "currency",
      currency: "IDR",
      maximumFractionDigits: 0,
    }).format(angka);
  };

  const isPemasukan = transaksi.jenis === "pemasukan";

  return (
    <li className={`item-transaksi ${isPemasukan ? "border-hijau" : "border-merah"}`}>
      <div className="info-transaksi">
        <span className="keterangan-transaksi">{transaksi.keterangan}</span>
        <span className="tanggal-transaksi">{transaksi.tanggal}</span>
      </div>

      <div className="nominal-dan-aksi">
        {/* Kriteria (d): Nominal pemasukan berwarna hijau dan pengeluaran berwarna merah */}
        <span className={`nominal-teks ${isPemasukan ? "teks-hijau" : "teks-merah"}`}>
          {isPemasukan ? `+ ${formatRupiah(transaksi.nominal)}` : `-
${formatRupiah(transaksi.nominal)}`}
        </span>

        {/* Kriteria (b): Tombol hapus */}
        <button
          className="tombol tombol-merah tombol-kecil"
          onClick={() => onHapus(transaksi.id)}
          title="Hapus transaksi ini"
        >
          Hapus
        </button>
      </div>
    </li>
  );
}

export default ItemTransaksi;
```

6. App.css

```css
/* Styling Utama / Wadah */
.wadah {
  max-width: 680px;
  margin: 50px auto;
  padding: 0 20px;
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
  color: #1F2937;
  background-color: #F8FAFC;
  min-height: 100vh;
}
```

```
/* Judul & Subjudul */
.judul {
  font-size: 28px;
  font-weight: 800;
  color: #0F172A;
  margin-bottom: 6px;
  letter-spacing: -0.5px;
}

.subjudul {
  color: #64748B;
  font-size: 15px;
  margin-top: 0;
  margin-bottom: 28px;
}

/* Komponen Kartu yang Lebih Elegan */
.kartu {
  background: #FFFFFF;
  border: 1px solid #E2E8F0;
  border-radius: 16px;
  padding: 24px;
  margin-bottom: 24px;
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.04), 0 4px 6px -4px rgba(0, 0, 0, 0.02);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.kartu:hover {
  box-shadow: 0 20px 25px -5px rgba(0, 0, 0, 0.06), 0 8px 10px -6px rgba(0, 0, 0, 0.04);
}

.kartu h2 {
  margin-top: 0;
  font-size: 18px;
  font-weight: 700;
  color: #1E293B;
  margin-bottom: 16px;
}

/* Baris & Input Form */
.baris {
```

```
  display: flex;
  gap: 10px;
}

.input {
  flex: 1;
  padding: 12px 16px;
  border: 1.5px solid #CBD5E1;
  border-radius: 10px;
  font-size: 14px;
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
  background-color: #F8FAFC;
}

.input:focus {
  border-color: #3B82F6;
  background-color: #FFFFFF;
  box-shadow: 0 0 0 4px rgba(59, 130, 246, 0.1);
}

/* Desain Tombol Interaktif */
.tombol {
  padding: 12px 20px;
  border: none;
  border-radius: 10px;
  background: #3B82F6;
  color: #FFFFFF;
  font-weight: 600;
  font-size: 14px;
  cursor: pointer;
  transition: background 0.2s, transform 0.1s, box-shadow 0.2s;
  box-shadow: 0 4px 6px -1px rgba(59, 130, 246, 0.2);
}

.tombol:hover {
  background: #2563EB;
  box-shadow: 0 6px 8px -1px rgba(59, 130, 246, 0.3);
}

.tombol:active {
```

```
  transform: scale(0.98);
}

.tombol:disabled {
  background: #E2E8F0;
  color: #94A3B8;
  cursor: not-allowed;
  box-shadow: none;
}

.tombol-merah {
  background: #EF4444;
  box-shadow: 0 4px 6px -1px rgba(239, 68, 68, 0.2);
}

.tombol-merah:hover {
  background: #DC2626;
  box-shadow: 0 6px 8px -1px rgba(239, 68, 68, 0.3);
}

.tombol-kecil {
  padding: 6px 12px;
  font-size: 12px;
  border-radius: 8px;
}

/* Container & Tombol Filter */
.filter-container {
  display: flex;
  gap: 8px;
  margin: 16px 0;
  background: #F1F5F9;
  padding: 4px;
  border-radius: 12px;
}

.tombol-filter {
  flex: 1;
  background: transparent;
  border: none;
  padding: 8px 12px;
```

```
  font-size: 13px;
  font-weight: 600;
  color: #64748B;
  border-radius: 8px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.tombol-filter:hover {
  color: #1E293B;
}

.tombol-filter.aktif {
  background: #FFFFFF;
  color: #2563EB;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);
}

/* Daftar Tugas & Item */
.daftar {
  list-style: none;
  padding: 0;
  margin: 0;
}

.item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 14px;
  border-bottom: 1px solid #F1F5F9;
  border-radius: 10px;
  transition: background 0.2s;
}

.item:hover {
  background-color: #F8FAFC;
}

.item span {
  flex: 1;
```

```
  font-size: 14px;
  color: #334155;
  transition: color 0.2s;
}

.selesai {
  text-decoration: line-through;
  color: #94A3B8 !important;
}

/* Status Kosong & Galat */
.kosong {
  color: #94A3B8;
  text-align: center;
  font-style: italic;
  padding: 24px 0;
  font-size: 14px;
}

.galat {
  color: #EF4444;
  font-size: 13px;
  margin: 6px 0 0;
  font-weight: 500;
}

.ringkasan {
  font-size: 14px;
  color: #64748B;
  font-weight: 500;
  margin-bottom: 12px;
}

/* Tombol Hapus Semua Selesai */
.tombol-hapus-semua {
  width: 100%;
  margin-top: 16px;
  background: #FEF2F2;
  color: #DC2626;
  border: 1px dashed #FCA5A5;
  padding: 10px;
```

```
  border-radius: 10px;
  font-weight: 600;
  font-size: 13px;
  cursor: pointer;
  transition: background 0.2s;
}

.tombol-hapus-semua:hover {
  background: #FEE2E2;
}

/* ========================================
   Styling Navigasi Tab Modul
   ======================================== */
.tab-navigasi {
  display: flex;
  gap: 10px;
  margin-bottom: 24px;
  background: #E2E8F0;
  padding: 6px;
  border-radius: 14px;
}

.tombol-tab {
  flex: 1;
  background: transparent;
  border: none;
  padding: 10px 16px;
  font-size: 14px;
  font-weight: 700;
  color: #475569;
  border-radius: 10px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.tombol-tab:hover {
  color: #0F172A;
}

.tombol-tab.aktif {
```

```
  background: #FFFFFF;
  color: #2563EB;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.08);
}

/* ========================================
   Styling Tugas 3: Catatan Keuangan Mini
   ======================================== */
.modul-keuangan {
  display: flex;
  flex-direction: column;
}

.form-keuangan {
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.form-baris {
  display: flex;
  gap: 12px;
}

.form-grup {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.form-grup label {
  font-size: 13px;
  font-weight: 600;
  color: #334155;
}

.tombol-simpan {
  margin-top: 8px;
  background-color: #059669;
  box-shadow: 0 4px 6px -1px rgba(5, 150, 105, 0.2);
}
```

```
.tombol-simpan:hover {
  background-color: #047857;
  box-shadow: 0 6px 8px -1px rgba(5, 150, 105, 0.3);
}

/* Ringkasan Saldo */
.kartu-ringkasan-saldo {
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.saldo-utama {
  text-align: center;
  padding: 18px;
  background: #F8FAFC;
  border: 1px solid #E2E8F0;
  border-radius: 14px;
}

.saldo-utama span {
  font-size: 13px;
  color: #64748B;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.saldo-utama h3 {
  font-size: 32px;
  margin: 6px 0 0;
  font-weight: 800;
}

.detail-arus-kas {
  display: flex;
  gap: 12px;
}

.box-arus {
```

```
  flex: 1;
  padding: 14px;
  border-radius: 12px;
  display: flex;
  flex-direction: column;
  gap: 4px;
}

.box-arus.pemasukan {
  background-color: #ECFDF5;
  border: 1px solid #A7F3D0;
}

.box-arus.pengeluaran {
  background-color: #FEF2F2;
  border: 1px solid #FECACA;
}

.box-arus span {
  font-size: 12px;
  color: #64748B;
  font-weight: 500;
}

.box-arus strong {
  font-size: 17px;
  font-weight: 700;
}

/* Kriteria (d): Nominal Pemasukan Hijau & Pengeluaran Merah */
.teks-hijau {
  color: #10B981 !important;
}

.teks-merah {
  color: #EF4444 !important;
}

/* Daftar & Item Transaksi */
.daftar-transaksi {
  list-style: none;
```

```
  padding: 0;
  margin: 0;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.item-transaksi {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 16px;
  background-color: #F8FAFC;
  border-radius: 10px;
  border: 1px solid #E2E8F0;
  transition: transform 0.15s ease, box-shadow 0.15s ease;
}

.item-transaksi:hover {
  transform: translateY(-1px);
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}

.item-transaksi.border-hijau {
  border-left: 5px solid #10B981;
}

.item-transaksi.border-merah {
  border-left: 5px solid #EF4444;
}

.info-transaksi {
  display: flex;
  flex-direction: column;
  gap: 2px;
}

.keterangan-transaksi {
  font-weight: 600;
  color: #1E293B;
  font-size: 14px;
```

```
}

.tanggal-transaksi {
  font-size: 12px;
  color: #94A3B8;
}

.nominal-dan-aksi {
  display: flex;
  align-items: center;
  gap: 14px;
}

.nominal-teks {
  font-weight: 700;
  font-size: 15px;
}

/* Kriteria (e): Pesan Khusus Transaksi Kosong */
.pesan-kosong-keuangan {
  text-align: center;
  padding: 32px 16px;
  background-color: #F8FAFC;
  border: 1.5px dashed #CBD5E1;
  border-radius: 12px;
  color: #64748B;
}

.pesan-kosong-keuangan p {
  margin: 0;
  font-weight: 600;
  font-size: 15px;
  color: #334155;
}

.pesan-kosong-keuangan small {
  display: block;
  margin-top: 4px;
  color: #94A3B8;
  font-size: 13px;
}
```

Bagian 2. Penjelasan Kode

2.1 Modul Daftar Tugas: Filter Dinamis (Tugas 1)

Pada berkas App.jsx, fitur filter diimplementasikan dengan menambahkan state baru const [filter, setFilter] = useState("semua"). Nilai state ini dapat bernilai 'semua', 'belum', atau 'selesai'. Untuk menampilkan data yang sesuai pilihan tanpa merusak state utama daftarTugas, diterapkan konsep *derived state* melalui variabel daftarTampil:

jsx

```
const daftarTampil = daftarTugas.filter((tugas) => {
  if (filter === "belum") return !tugas.selesai;
  if (filter === "selesai") return tugas.selesai;
  return true;
});
```

Pada bagian antarmuka tombol, indikator tombol yang sedang aktif diatur menggunakan class name dinamis berbasis ekspresi *template literals* dan *ternary operator*:

jsx

```
className={`tombol-filter ${filter === "semua" ? "aktif" : ""}`}
```

2.2 Modul Daftar Tugas: Conditional Rendering Hapus Selesai (Tugas 2)

Tombol pembersih tugas selesai diatur agar hanya muncul apabila ada minimal satu tugas yang berstatus selesai: true. Jumlah tugas selesai dihitung menggunakan jumlahSelesai = daftarTugas.filter((tugas) => tugas.selesai).length. Untuk merendernya, digunakan *short-circuit evaluation* operator logika &&:

jsx

```
{jumlahSelesai > 0 && (
  <button className="tombol-hapus-semua" onClick={hapusTugasSelesai}>
    Hapus Semua yang Selesai ({jumlahSelesai})
  </button>
)}
```

Fungsi hapusTugasSelesai memperbarui state secara *immutable* dengan menyaring dan mempertahankan hanya tugas-tugas yang belum selesai (!tugas.selesai).

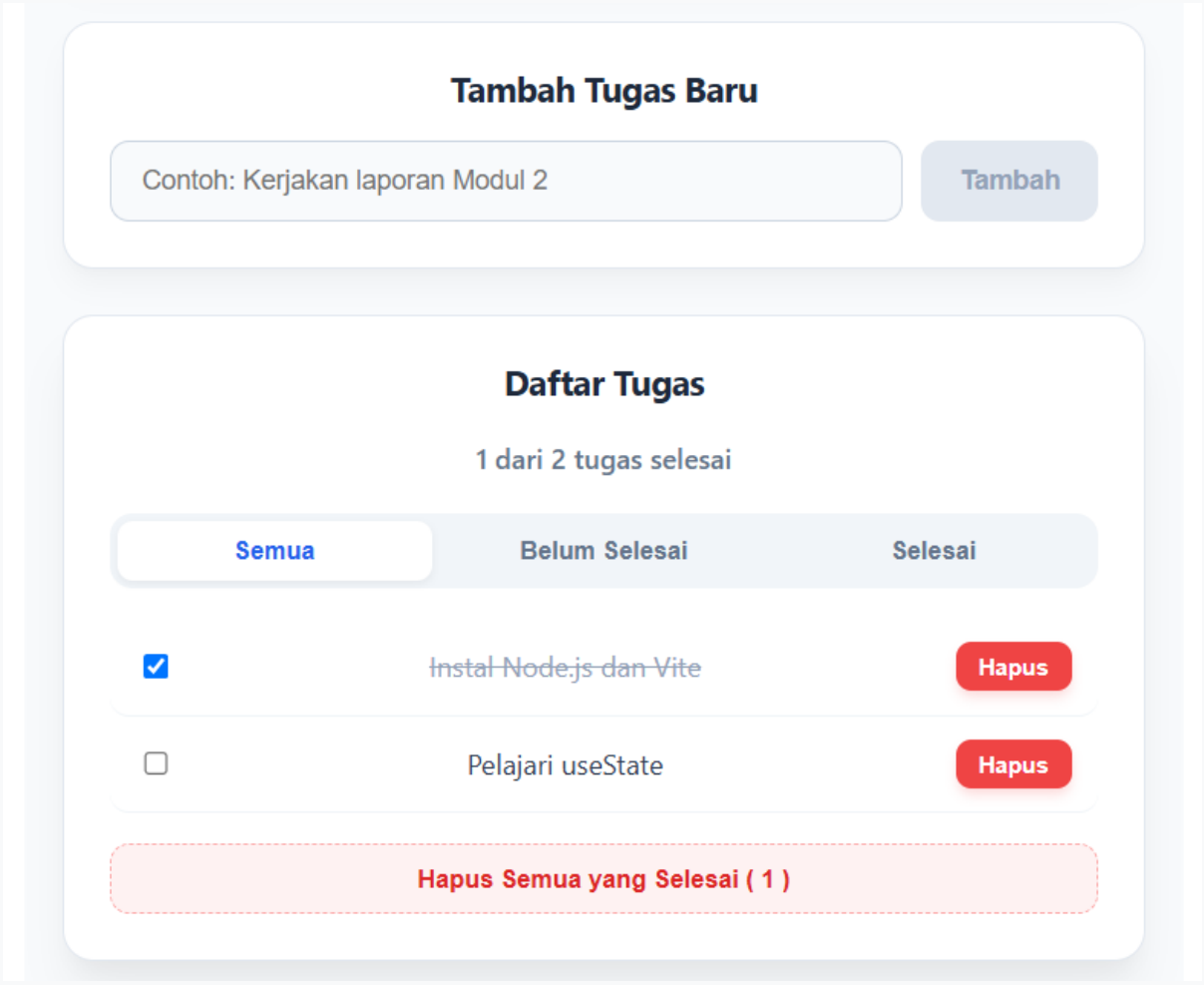
2.3 Modul Catatan Keuangan Mini (Tugas 3)

Aplikasi catatan keuangan mini dipecah menjadi beberapa komponen independen sesuai kriteria:

1. **FormTransaksi.jsx**: Berupa formulir terkendali (*controlled component*) yang mengikat nilai input ke dalam state lokal (keterangan, nominal, jenis). Sesuai petunjuk praktikum bahwa nilai input form selalu bertipe *string*, fungsi Number(nominal) digunakan untuk mengonversi nilai menjadi tipe data numerik sebelum disimpan.
2. **RingkasanSaldo.jsx**: Menghitung kalkulasi saldo bersih secara otomatis (*derived state*) tanpa perlu membuat state tambahan. Fungsi .reduce() digunakan untuk menjumlahkan total pemasukan dan total pengeluaran, yang kemudian dikurangkan untuk memperoleh total saldo akhir.
3. **ItemTransaksi.jsx**: Menampilkan item baris transaksi dengan penanda warna dinamis (hijau untuk jenis pemasukan dan merah untuk pengeluaran) melalui class teks-hijau, teks-merah, serta border penanda warna. Komponen ini juga menyertakan tombol aksi untuk menghapus transaksi berdasarkan ID.
4. **DaftarTransaksi.jsx**: Merender kumpulan komponen ItemTransaksi melalui pemetaan .map(). Dilengkapi *conditional rendering* ternary untuk menampilkan kotak pesan pemberitahuan informatif jika array data transaksi masih kosong.
5. **CatatanKeuangan.jsx & Sinkronisasi localStorage**: Komponen kontainer yang memuat state transaksi. Data dimuat saat inisialisasi awal melalui localStorage.getItem("catatan-keuangan-mini"), dan setiap terjadi perubahan state (penambahan/penghapusan data), Hook useEffect secara otomatis menyimpan data terbaru ke dalam localStorage menggunakan localStorage.setItem().

Bagian 3. Screenshot Hasil Antarmuka

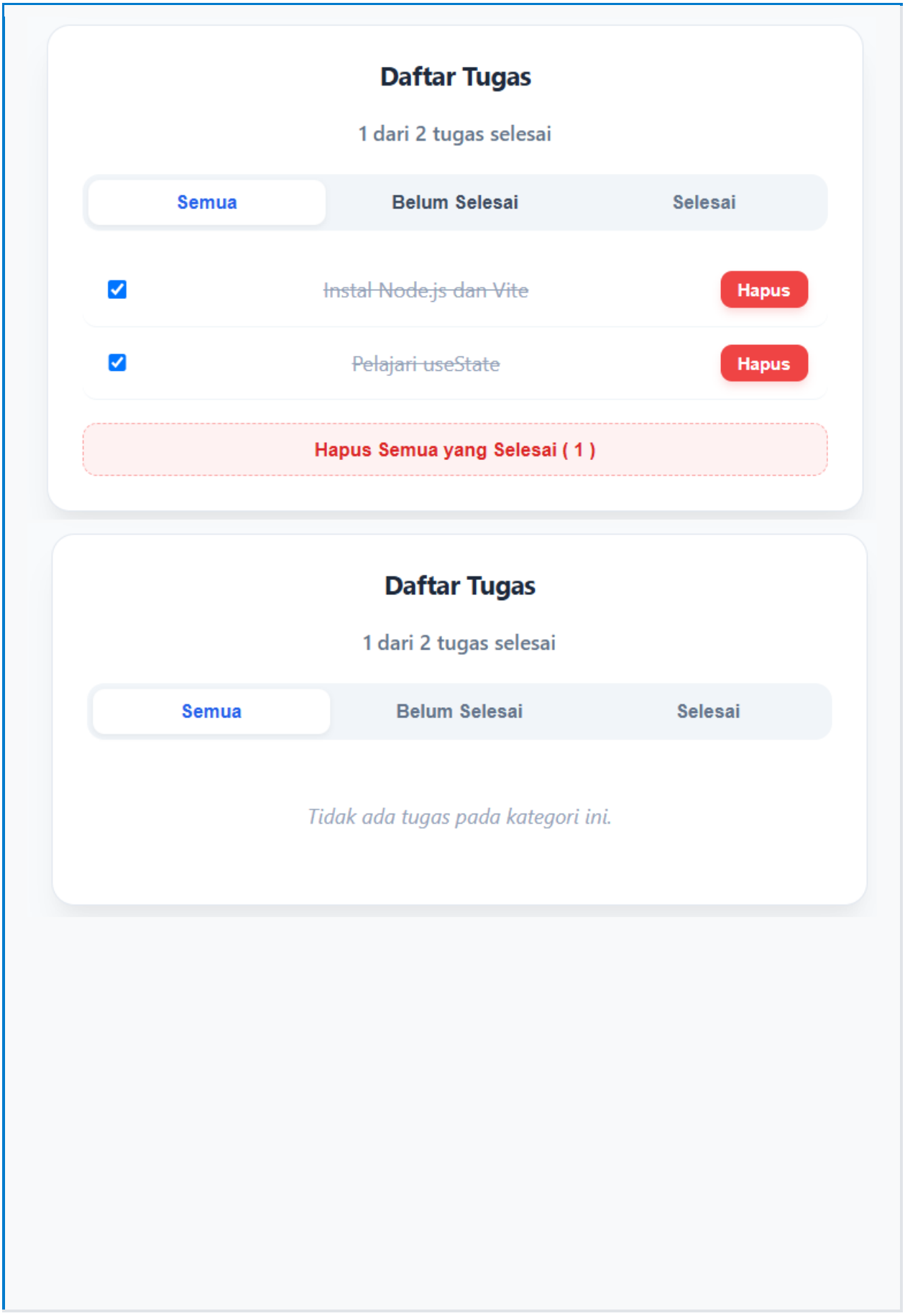
1. **Tab Daftar Tugas — Filter "Semua" Akti**



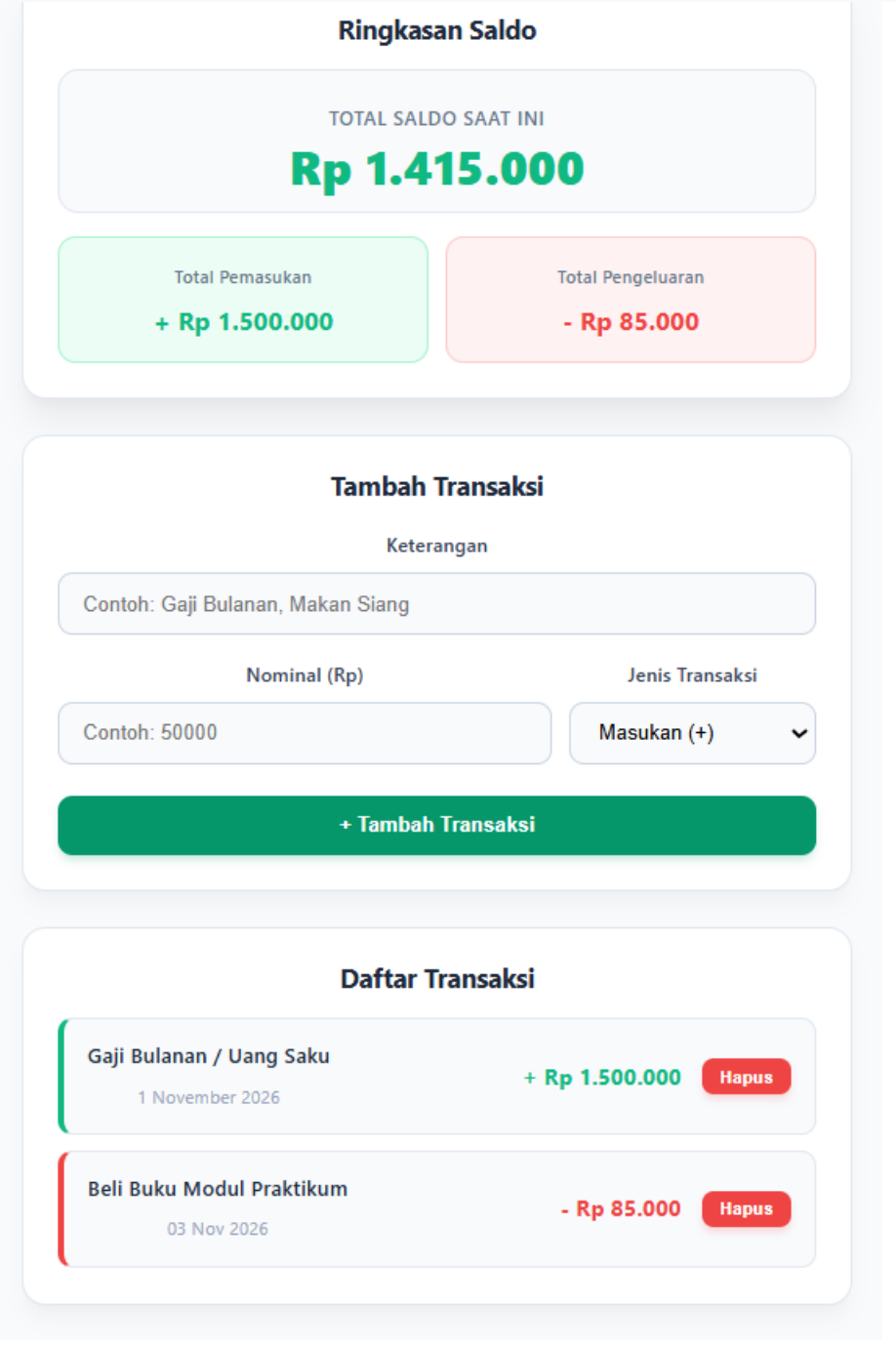
2. **Tab Daftar Tugas — Pengujian Filter "Belum Selesai" dan "Selesai"**



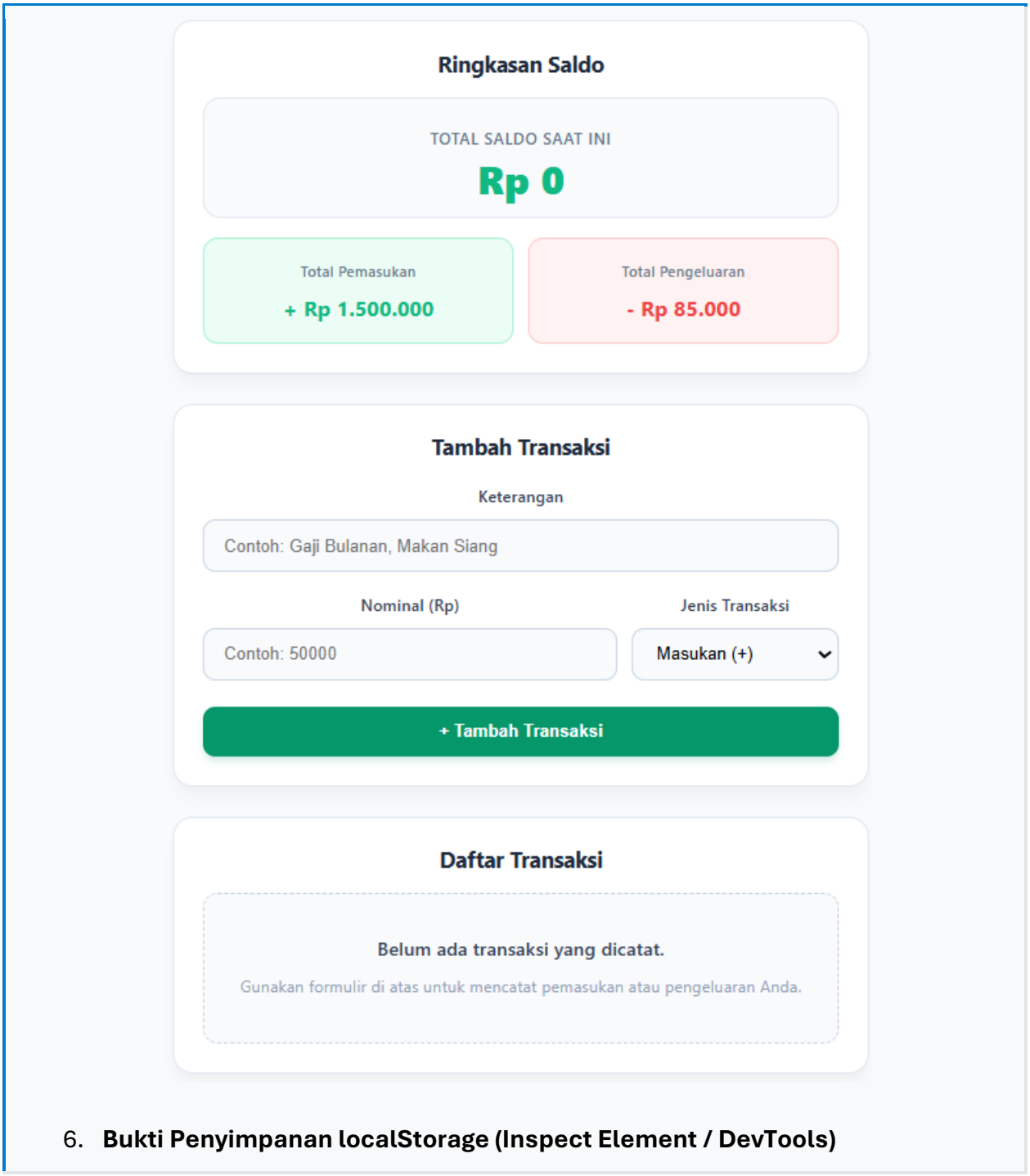
3. **Conditional Rendering Tombol "Hapus Semua yang Selesai"**


Daftar Tugas
1 dari 2 tugas selesai
Semua
Belum Selesai
Selesai
Instal Node.js dan Vite
Hapus
Pelajari useState
Hapus
Hapus Semua yang Selesai (1)
Daftar Tugas
1 dari 2 tugas selesai
Semua
Belum Selesai
Selesai
Tidak ada tugas pada kategori ini.

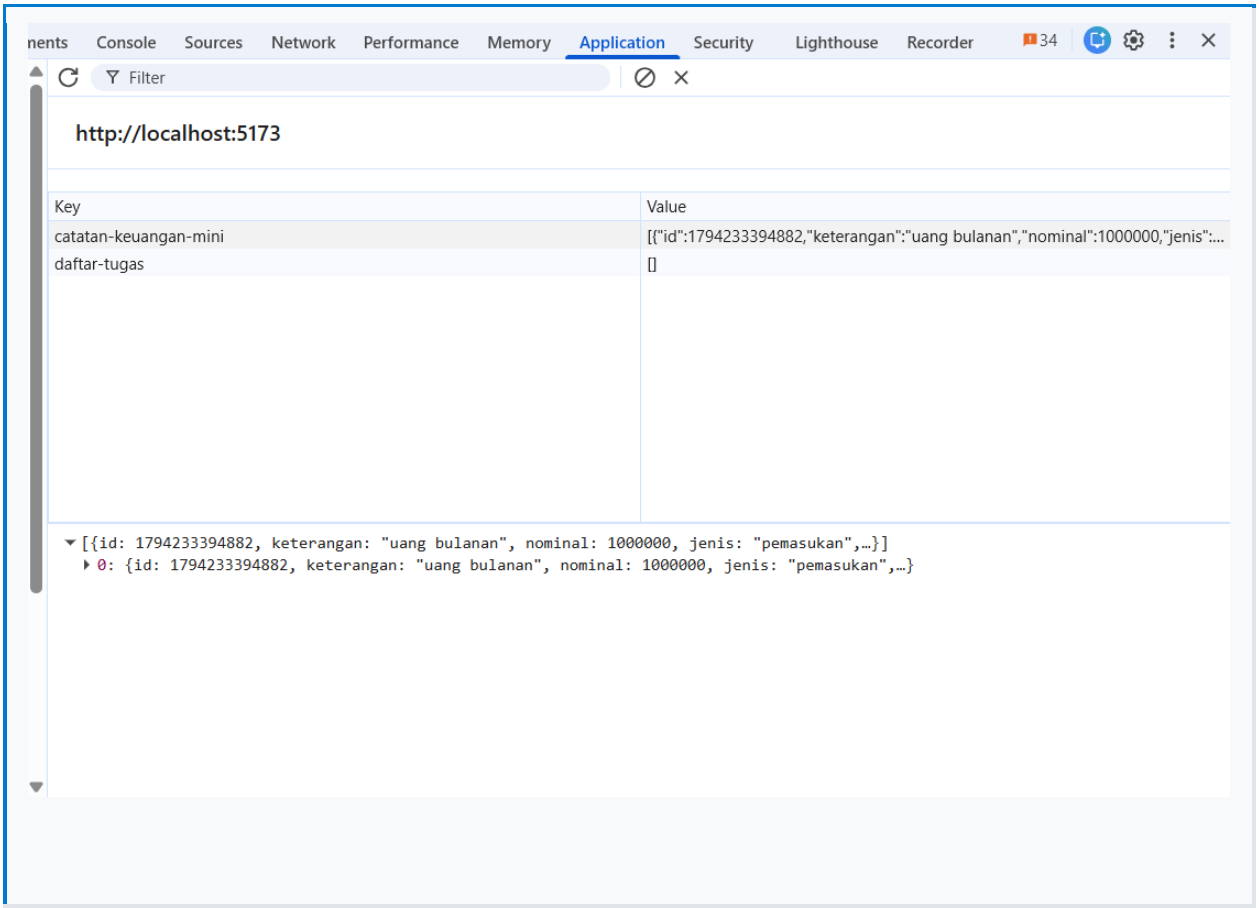
4. **Catatan Keuangan — Antarmuka Utama & Saldo Otomatis**



5. **Tab Catatan Keuangan — Pesan Khusus Ketika Transaksi Kosong**



6. **Bukti Penyimpanan localStorage (Inspect Element / DevTools)**



Bagian 4. Penjelasan Hasil Antarmuka

4.1 Filter Kategori Tugas (Gambar 3.1 & Gambar 3.2)
Antarmuka daftar tugas menampilkan bilah kontrol filter dengan tiga pilihan tombol. Ketika pengguna menekan salah satu tombol, tombol tersebut otomatis berubah warna latar belakangnya menjadi putih dengan teks kontras untuk menandakan status aktif. Daftar tugas di bawahnya merespons seketika: saat memilih "Belum Selesai", hanya item yang belum dicentang yang terlihat; saat memilih "Selesai", hanya item dengan coretan dan tanda centang yang ditampilkan.

4.2 Tombol Pembersih Tugas Bersyarat (Gambar 3.3)
Pada antarmuka daftar tugas, tombol bergaris putus-putus merah "Hapus Semua yang Selesai" tampil di bawah daftar hanya jika terdapat tugas yang selesai. Saat diklik, seluruh tugas yang telah selesai dihapus dari daftar seketika, dan tombol tersebut langsung menghilang secara otomatis dari layar karena kondisi jumlahSelesai > 0 sudah tidak terpenuhi.

4.3 Ringkasan Arus Kas dan Pewarnaan Nominal (Gambar 3.4)
Pada tab "Catatan Keuangan", kartu bagian atas menyajikan informasi total sisa saldo dengan ukuran font besar dan kontras. Nominal arus kas dibedakan secara visual: pemasukan ditandai dengan teks warna hijau terang (#10B981) dan prefiks tanda tambah (+), sedangkan pengeluaran ditandai dengan warna merah tegas (#EF4444) dan

prefiks tanda kurang (-). Setiap baris transaksi memiliki strip border samping yang selaras dengan jenis transaksinya.

4.4 Kondisi Data Kosong (Gambar 3.5)

Ketika pengguna menghapus semua transaksi yang ada pada daftar, antarmuka tidak menampilkan tabel/daftar yang kosong melompong, melainkan memunculkan sebuah wadah bergaris putus-putus (*dashed box*) dengan teks ramah yang memberitahukan bahwa transaksi belum tersedia dan mengajak pengguna untuk mulai mencatat transaksi.

4.5 Bukti Persistensi Data LocalStorage (Gambar 3.6)

Panel DevTools peramban web pada tab Application memperlihatkan bahwa setiap tugas dan riwayat transaksi yang dimasukkan telah tersimpan dalam bentuk string JSON yang valid di penyimpanan lokal peramban. Saat halaman web dimuat ulang (*refresh*), seluruh data tidak hilang dan kembali terisi secara otomatis ke dalam komponen.